

NiceGUI 中 ui.run 的详细解析

`ui.run()` 是 NiceGUI 应用的**启动入口**，负责初始化 Web 服务器、加载配置、启动事件循环，并将 UI 组件挂载到服务器上。它基于 FastAPI（后端）和 Quasar（前端）封装，隐藏了底层复杂的服务器配置，同时提供丰富的参数满足定制化需求。以下从核心功能、参数详解、启动模式、进阶用法等维度全面解析：

一、核心作用

`ui.run()` 的核心职责：

- 初始化 FastAPI 应用实例（NiceGUI 的 `app` 对象底层关联 FastAPI 实例）；
- 配置 Web 服务器（默认 Uvicorn，也支持其他 ASGI 服务器）；
- 绑定 UI 组件、路由（`app.urls`）到服务器；
- 启动异步事件循环，监听指定端口并处理客户端请求；
- 管理应用的生命周期（启动 / 停止、热重载、多线程 / 进程等）。

二、基础用法

最简化的启动方式（默认配置）：

```
from nicegui import ui

ui.label('Hello NiceGUI!')
ui.button('Click me', on_click=lambda: ui.notify('Button clicked!'))

# 启动应用（所有配置使用默认值）
ui.run()
```

执行后会自动：

- 启动 Uvicorn 服务器，监听 `localhost:8080`；
- 自动打开默认浏览器，访问 `http://localhost:8080`；
- 控制台输出服务器启动日志（如端口、访问地址）。

三、核心参数详解

`ui.run()` 提供数十个参数，覆盖服务器、网络、前端、生命周期等维度，核心参数如下：

参数名	类型	默认值	核心作用
<code>host</code>	str	<code>'localhost'</code>	绑定的服务器地址：- <code>'localhost'</code> ：仅本机可访问；- <code>'0.0.0.0'</code> ：局域网 / 公网可访问；- 特定 IP（如 <code>'192.168.1.100'</code> ）：仅绑定该 IP。
<code>port</code>	int	<code>8080</code>	监听的端口号，若端口被占用会自动递增（如 8081、8082）。
<code>title</code>	str	<code>'NiceGUI'</code>	浏览器标签页标题。

参数名	类型	默认值	核心作用
favicon	str	None	浏览器标签页图标，支持本地路径（如 './favicon.ico'）或 URL。
reload	bool	False	热重载：修改代码后自动重启服务器（开发环境推荐开启）。
show	bool	True	启动后自动打开默认浏览器访问应用。
on_startup	callable	None	应用启动时执行的回调函数（如初始化数据库、加载配置）。
on_shutdown	callable	None	应用停止时执行的回调函数（如关闭数据库连接、保存数据）。
storage_secret	str	None	用于加密 app.storage（用户 / 会话存储）的密钥，生产环境必须设置。
uvicorn_logging_level	str	'warning'	Uvicorn 日志级别：'debug' / 'info' / 'warning' / 'error'。
workers	int	1	启动的 Uvicorn 工作进程数（仅生产环境推荐 > 1，需关闭 reload）。
api_base_url	str	'/api'	NiceGUI 内置 API 的基础路径（避免与自定义路由冲突）。
mount_path	str	'/'	应用挂载的根路径（如 '/myapp'，则访问地址为 http://localhost:8080/myapp）。

关键参数示例

1. 允许局域网访问 + 自定义端口 + 热重载

```
ui.run(
    host='0.0.0.0', # 局域网内其他设备可访问
    port=9000,      # 自定义端口
    reload=True     # 开发模式：修改代码自动重启
)
```

2. 生产环境配置（多进程 + 日志 + 存储密钥）

```
ui.run(
    host='0.0.0.0',
    port=80,
    reload=False,
    workers=4, # 4个工作进程（根据CPU核心数调整）
    uvicorn_logging_level='info', # 输出info级别日志
    storage_secret='my_secure_secret_123', # 加密存储
    show=False # 生产环境不自动打开浏览器
)
```

3. 启动 / 停止回调

```
def on_start():
    print('应用启动成功！初始化数据库...')
    # 执行数据库连接、配置加载等操作

def on_stop():
    print('应用停止！关闭数据库连接...')
    # 执行资源释放、数据保存等操作

ui.run(
    on_startup=on_start,
    on_shutdown=on_stop
)
```

四、启动模式与底层机制

1. 底层依赖

NiceGUI 基于 FastAPI (ASGI 框架) 构建, `ui.run()` 默认使用 Uvicorn (ASGI 服务器) 启动, 核心流程:

```
ui.run() → 初始化FastAPI实例 → 注册NiceGUI的路由/中间件 → 启动Uvicorn服务器 → 监听请求
```

2. 两种启动模式

(1) 阻塞模式 (默认)

`ui.run()` 会阻塞主线程, 直到手动停止 (如按 `Ctrl+C`), 适用于单应用场景。

(2) 非阻塞模式 (后台启动)

通过 `start()` 和 `stop()` 方法实现后台启动, 适用于集成到其他应用 (如定时任务、多应用管理):

```
from nicegui import ui

ui.label('后台运行的NiceGUI应用')

# 后台启动 (非阻塞)
server = ui.run(startup_mode='threaded') # 启动在子线程

# 业务逻辑: 执行其他代码
print('NiceGUI应用已后台启动, 继续执行主线程代码...')

# 手动停止服务器
# server.stop()
```

- `startup_mode` 支持:
 - `'threaded'`: 启动在子线程 (非阻塞);
 - `'subprocess'`: 启动在子进程 (隔离性更好);

- `'blocking'`: 默认阻塞模式。

五、常见问题与注意事项

1. 端口被占用

- 现象: 启动时提示 `Address already in use`;
- 解决:
 - 手动指定未被占用的端口 (如 `port=9001`) ;
 - 不指定端口, NiceGUI 会自动递增端口 (从 8080 开始) 。

2. 热重载失效

- 原因: `reload=True` 仅监控当前目录下的 `.py` 文件, 若修改其他目录的文件可能不触发重载;
- 解决: 通过 `reload_dirs` 参数指定监控目录:

```
ui.run(reload=True, reload_dirs=['./src', './templates']) # 监控src和templates目录
```

3. 生产环境部署

- 不建议直接用 `ui.run()` 部署, 推荐:
 1. 关闭 `reload` 和 `show`;
 2. 设置 `workers>1`;
 3. 配合 Nginx 反向代理 (处理静态文件、负载均衡) ;
 4. 使用 `systemd` / `supervisor` 管理进程 (防止应用意外退出) 。

4. 跨域问题

若前端与后端分离 (如前端部署在另一个域名), 需开启 CORS:

```
from fastapi.middleware.cors import CORSMiddleware

# 获取NiceGUI底层的FastAPI实例
fastapi_app = app.native.app

# 添加CORS中间件
fastapi_app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'], # 生产环境指定具体域名 (如['https://example.com'])
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

ui.run()
```

